

Operating System

Process and Threads



Contents

1. Program and process, C Program Execution

2. Process Related Details

- Process Structure
- Layout of a process in memory.
- Memory Layout of a C program
- Process States Diagram
- PCB
- Process scheduling Queues
- Process- Context Switching
- Schedulers

Process & Program

Process is a **program in execution**

Process execution must progress in sequential fashion

Program is a **passive entity** stored on disk (executable file), process is **active entity**

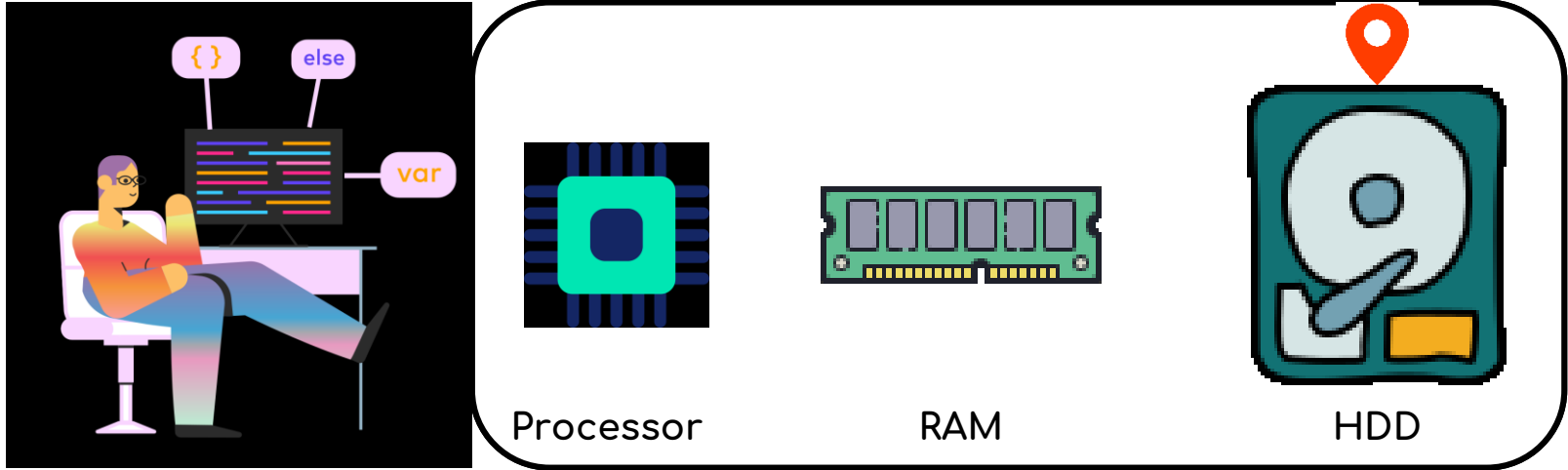
Program becomes process when executable file loaded into memory

Executable program can be loaded into memory

1. By mouse click
2. By command line

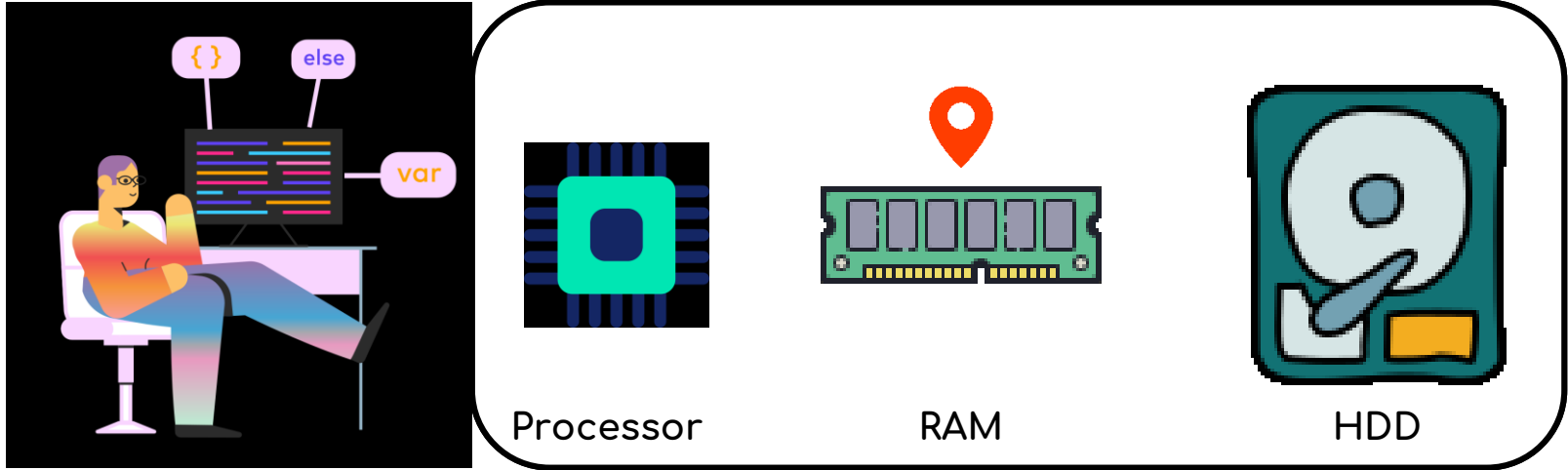
One program can have several processes.

Process



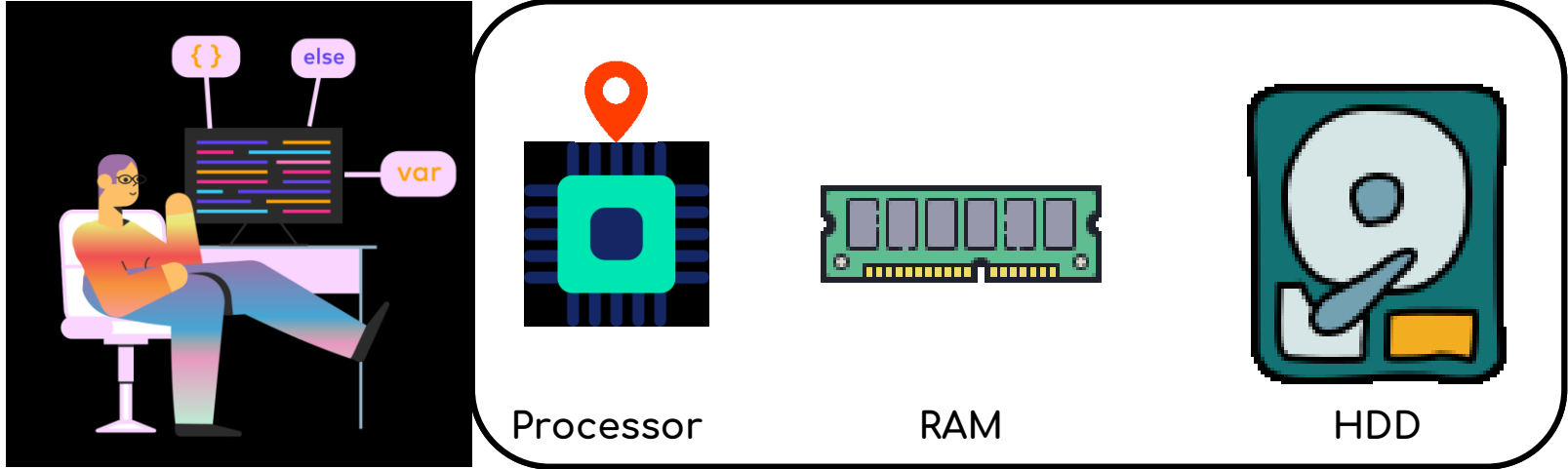
- A program is written using programming languages like Java, Python etc.
- The program is also referred to as source code
- After compilation of source code a byte code (executable) is created
- Bytecode is initially placed on disk like HDD/SSD

Process



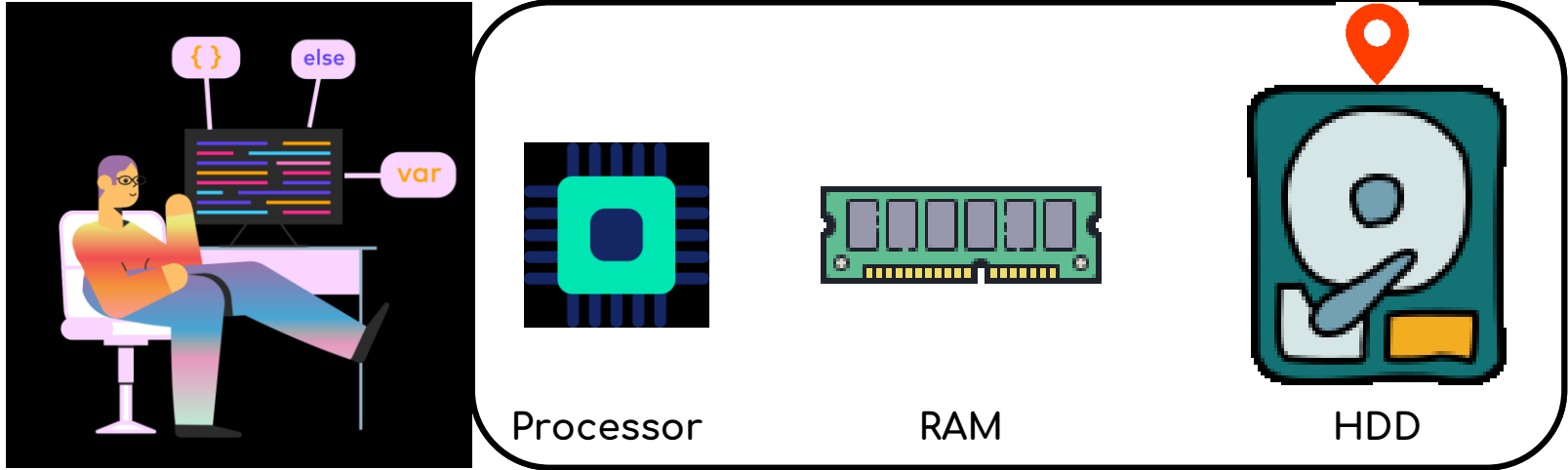
- When user run executable file using mouse or command line interface, executable and data related to program brought into RAM
- Memory gets allocated to code and data in RAM

Process



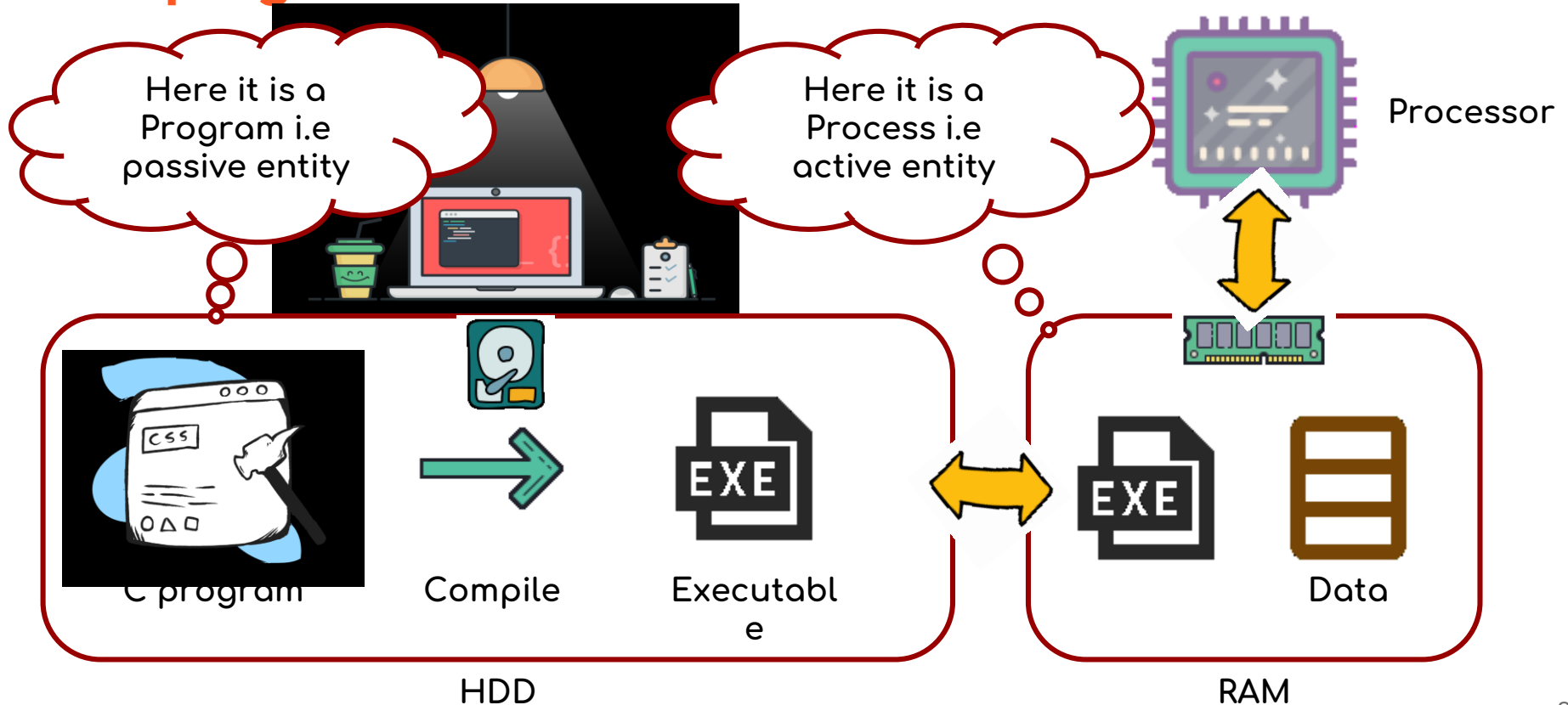
- Processor reads the data and instructions from memory and perform computations.
- Read/write happens on main memory.

Process



- After program execution source code and data related to code is removed from memory
- The executable remains on HDD/SSD

A C program execution



Process related details

- Process Structure
- Layout of a process in memory.
- Memory Layout of a C program
- Process States Diagram
- PCB
- Process scheduling Queues
- Process- Context Switching
- Schedulers

Process structure

```
1. Int a,b,c,Result
2. Int call_func(int a, int b)
3. {
4.     Int x = a*b
5.     Int y = a-b
6.     Return (y+x)
7. }
8. Int main()
9. {
10. Read a,b,c
11. Result = Call_func(a,b,c)
12. print(Results)
13. Return main
14. }
```

A process is program under execution.
A process is more than program code.

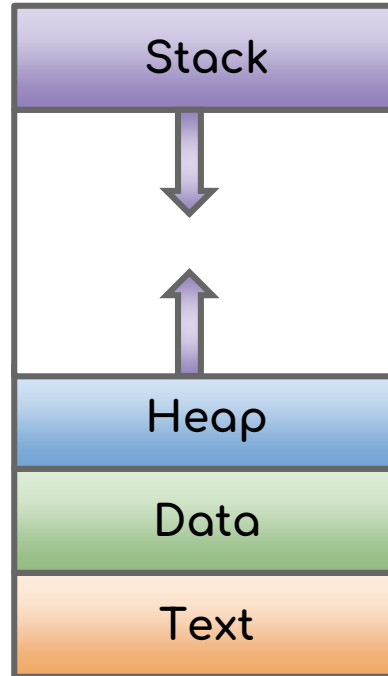
It also includes

- Process ID
- program counter
- registers
- state
- open files pointers
- Current directory, etc.

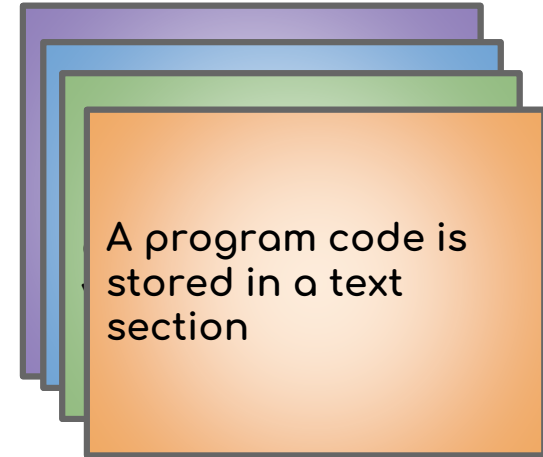
Layout of a process in memory.

```
1. Int a,b,c,Result
2. Int call_func(int a, int b)
3. {
4.   Int x = a*b
5.   Int y = a-b
6.   Return (y+x)
7. }
8. Int main()
9. {
10. Read a,b,c,Result
11. Result = Call_func(a,b)
12. print(Results)
13. Return main
14. }
```

Max

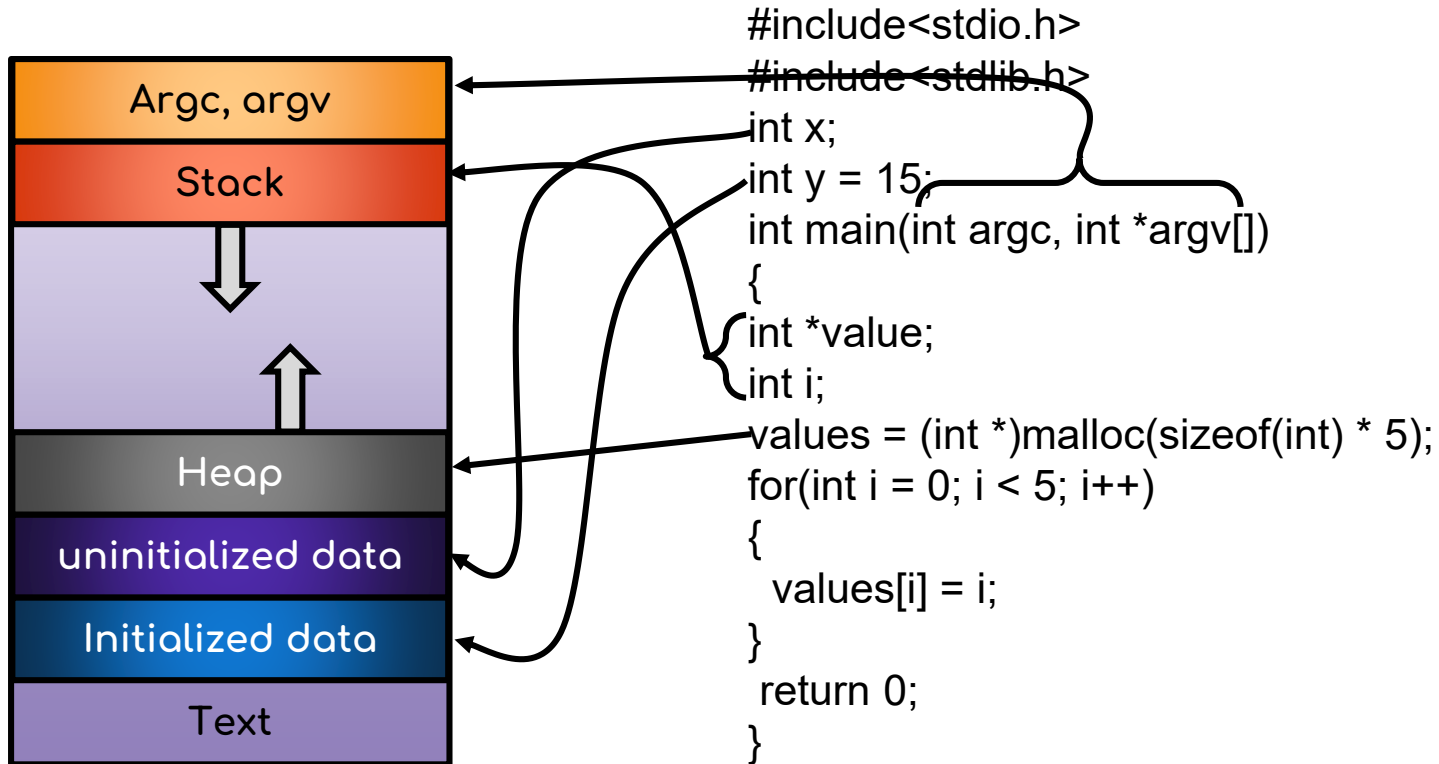


Heap and stack
grow or shrink

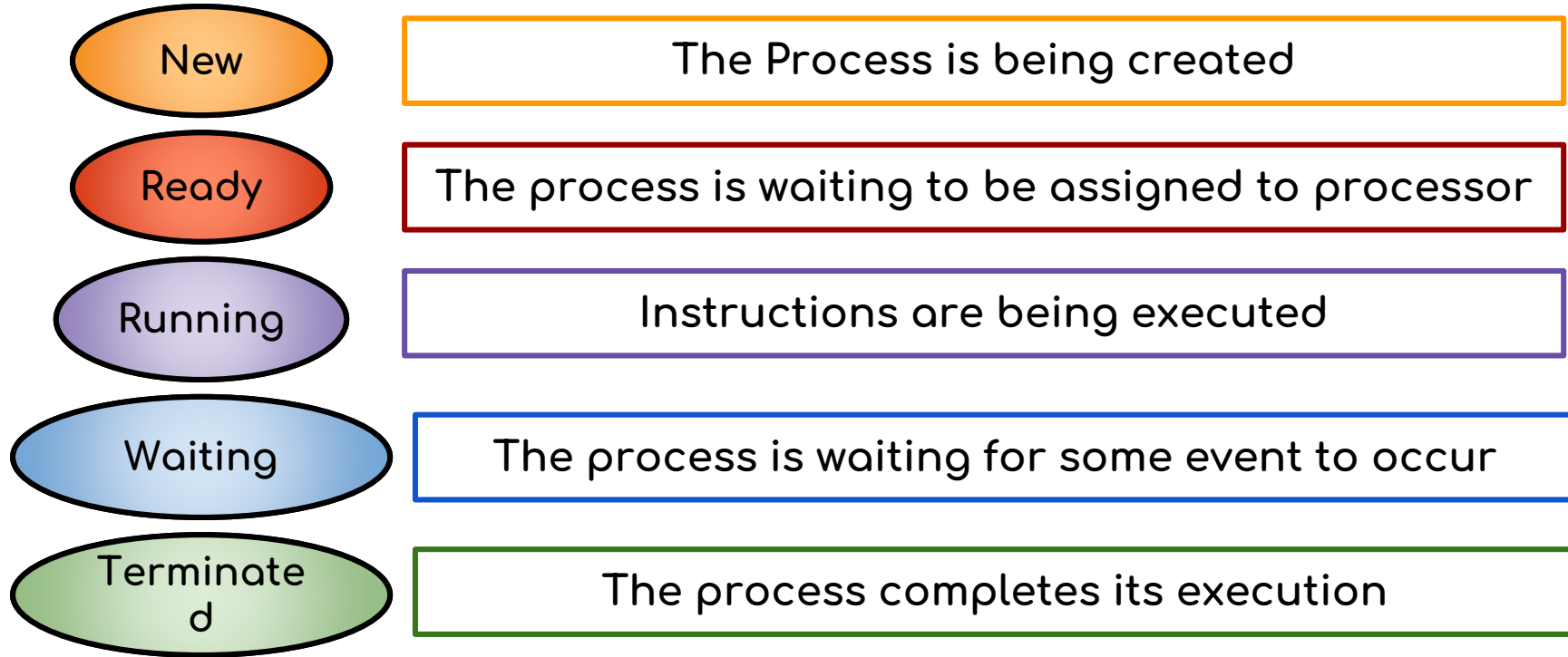


A program counter register keep track current instruction being executed and processor registers holds the variable values

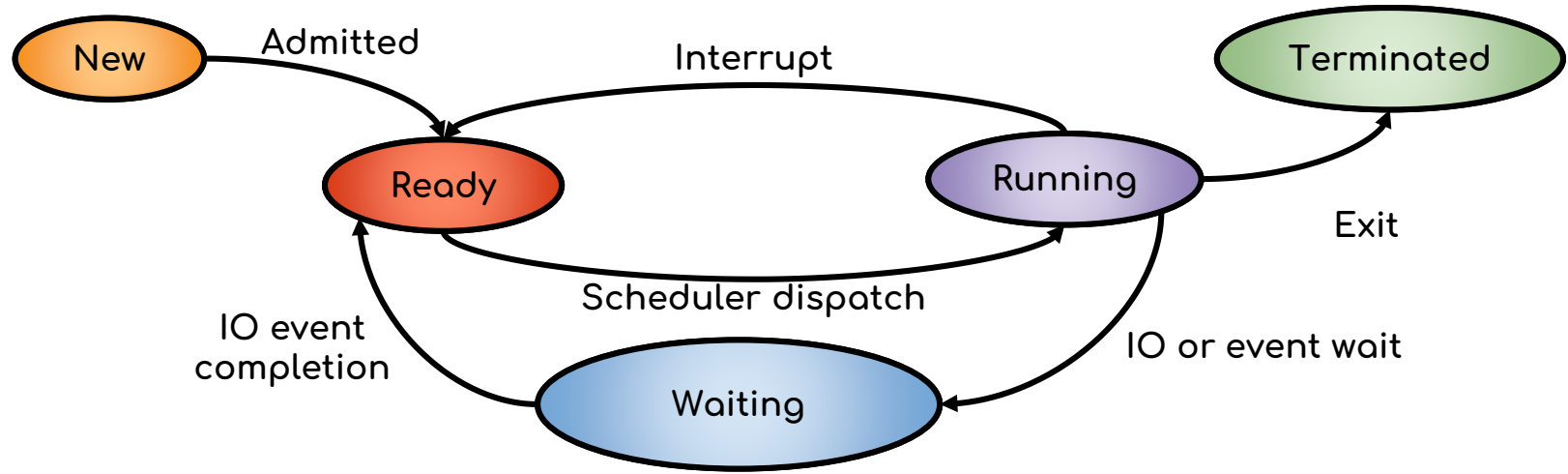
Memory layout of a C program



Process states



Process state diagram



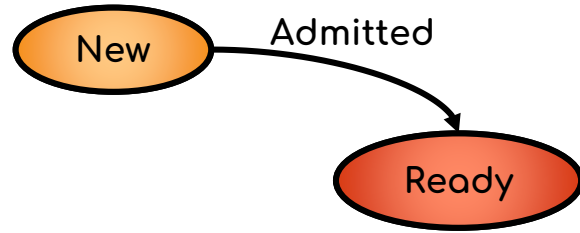
It is important to realize that only one process can be running on any processor core at any instant.

Process state diagram



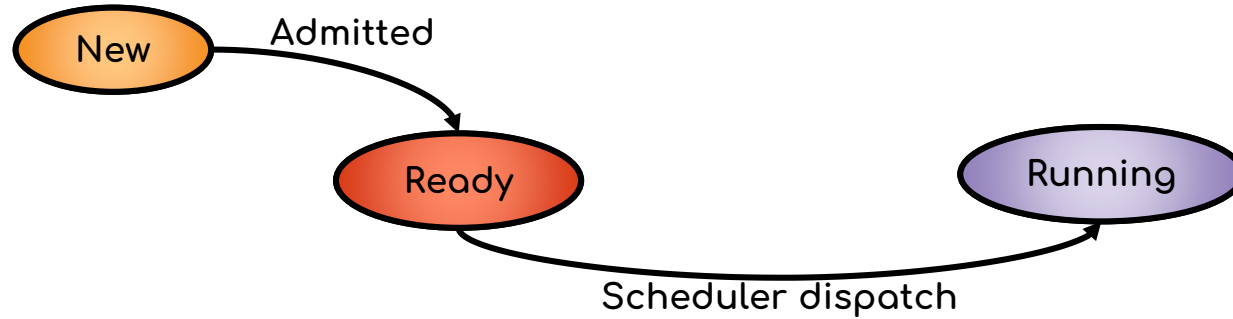
- When user clicks or run executable, data and source code of the program copied from disk into main memory after memory gets allocated.
- A process is just created and not assigned with any CPU resources

Process state diagram



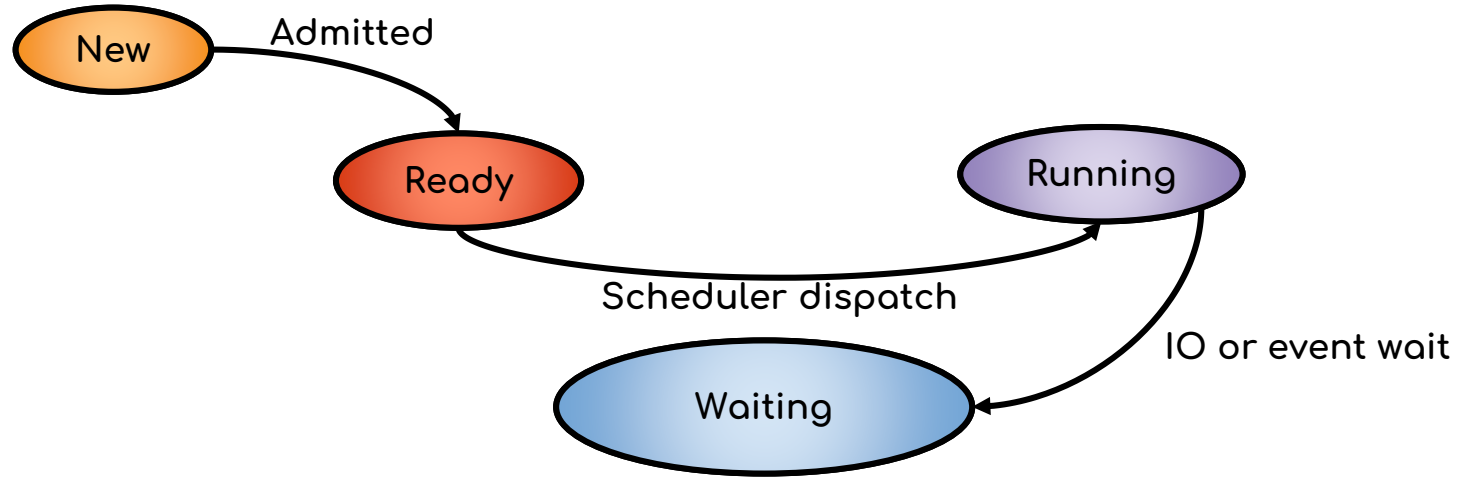
- After allocation of memory process is ready to execute
- Process is placed into ready queue by a **long term scheduler**
- A process will get chance to CPU resource based on **short term scheduler**
- Process may not get CPU for execution immediately as other process running on CPU

Process state diagram



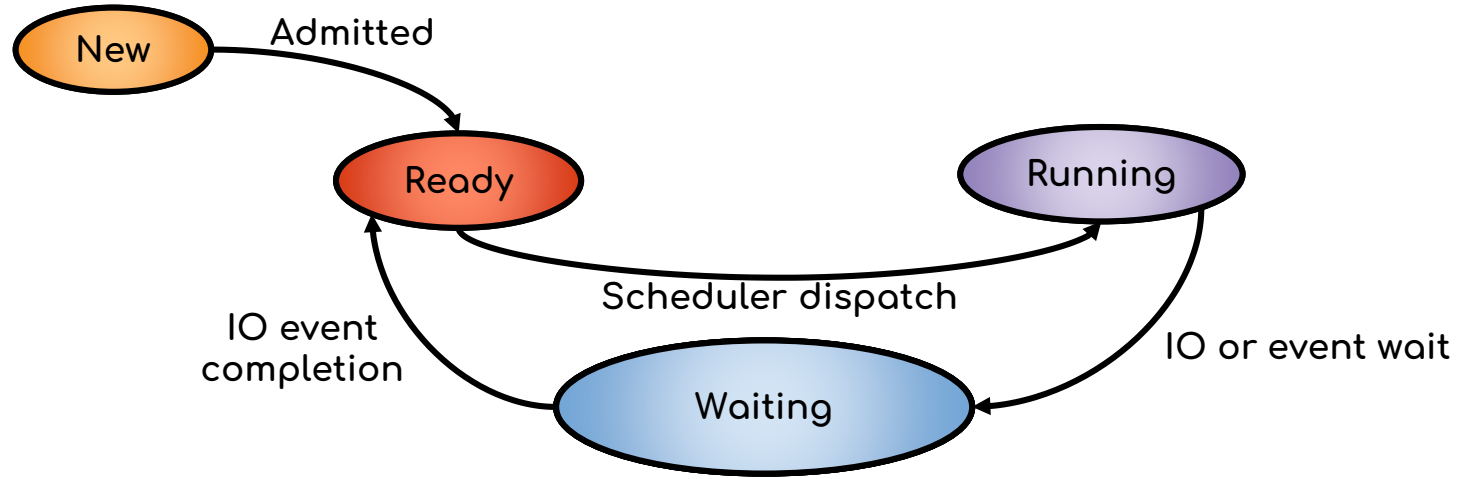
- When scheduler (short term) decides to execute the process, process moves to running state
- In running state process got the CPU and its executing

Process state diagram



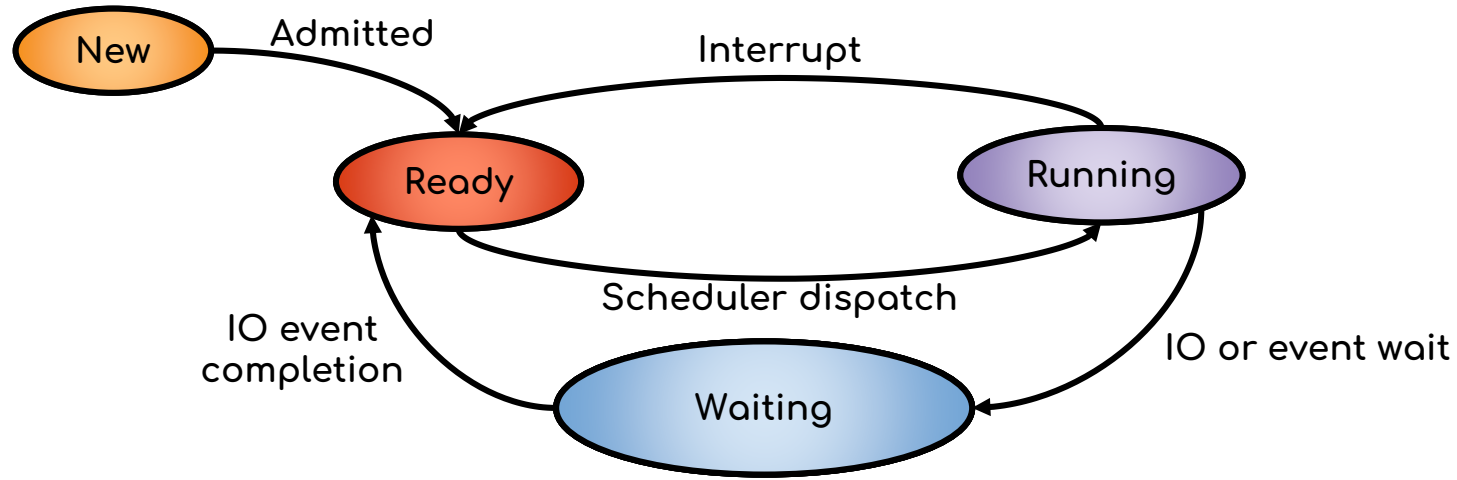
- During process execution process may expect IO operation such as read input from the user from keyboard
- Users are much slower as compared to processor
- Process should be kept in waiting state so that other processes get chance for CPU

Process state diagram



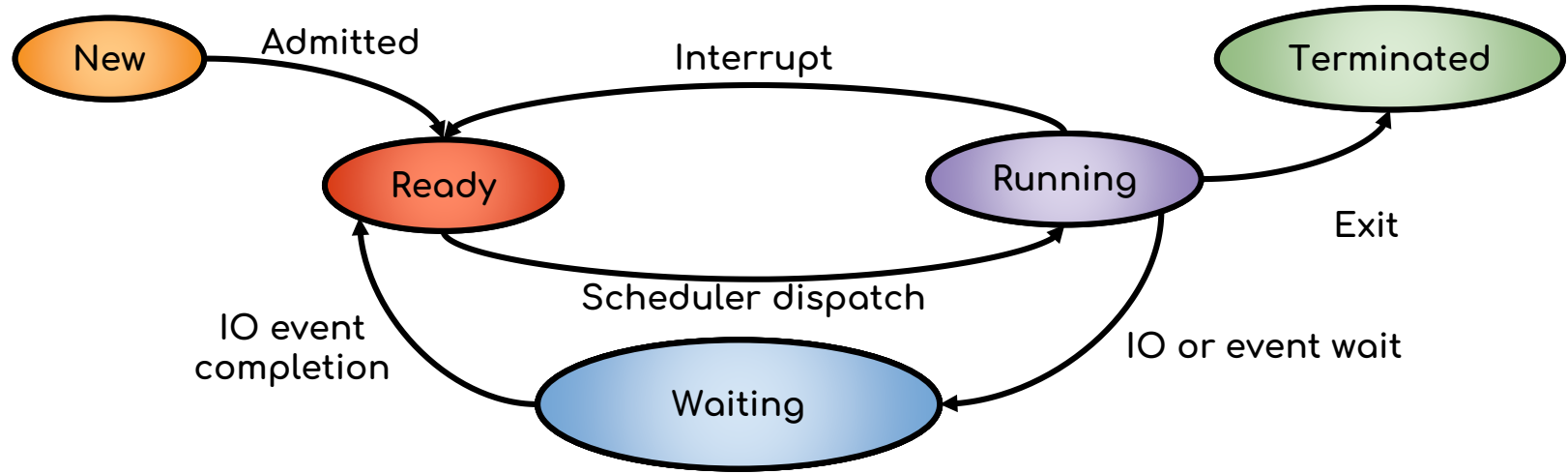
- On completion of IO, process will again kept in ready state
- Short term scheduler takes decision to keep process into running state again.

Process state diagram



- Sometimes process is given with time quanta to execute on CPU
- After completion of time quanta process gets interrupted and removed from running state to waiting state

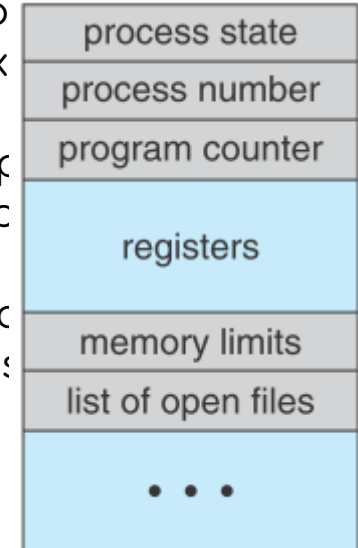
Process state diagram



- After completion of process execution it moves to terminated state

Process Control Block (PCB)

- Every process has Process Control Block (PCB) and Information associated with a process is stored in PCB
 1. **Process state** - Ready, running, terminated, etc.
 2. **Process ID** - Unique identification number assigned when process is created.
 3. **Program counter** - Location of the next instruction to be executed.
 4. **CPU registers** - Contents of all process centric registers.
 5. **CPU scheduling information** - Priorities, scheduling queue pointer.
 6. **Memory management information** - Memory allocation to process, data, stack and heap segment and their limits.
 7. **Accounting information** - CPU used, clock time elapsed since process started.
 8. **I/O status information** - I/O devices allocated to process, list of open files.



Process representation in Linux

- Represented by C structure `task_struct`

```
long state; /*denote state of the process */
```

```
struct sched entity se; /*denote scheduling information */
```

```
struct task_struct *parent; /*denotes this process's parent */
```

```
struct list_head children; /*denotes this process's children */
```

```
struct files_struct *files; /* denotes list of open files */
```

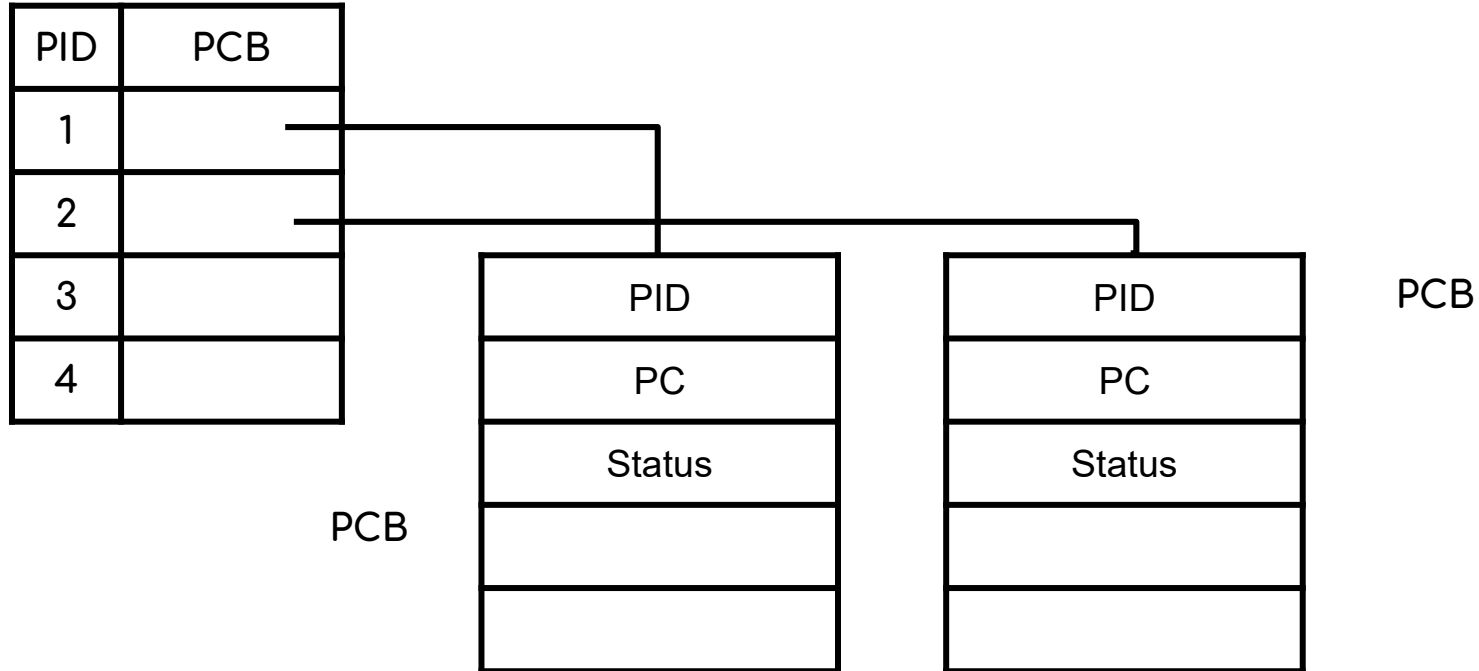
```
struct mm_struct *mm; /* denotes address space of this process */
```

```
struct task_struct *p_opptr,*p_pptr,*p_cptr,*p_ysptr,*p_osptr
```

```
/*denotes, op = original parent, p = parent, c = youngest child, ys = youngest sibling,  
os = older sibling */
```

Process representation in Linux

Process Table



Process Scheduling

- At any point of time number of processes are ready for the execution
- Operating system has to decide which process to be executed next
- The main objective of process scheduling is to maximize CPU utilization.
- Process “gives” up the CPU under two conditions, IO request and after N units of time have elapsed
- Once process “gives” up the CPU it is added to the ready queue
- Process scheduler selects among available processes in the ready queue for the next execution on CPU.

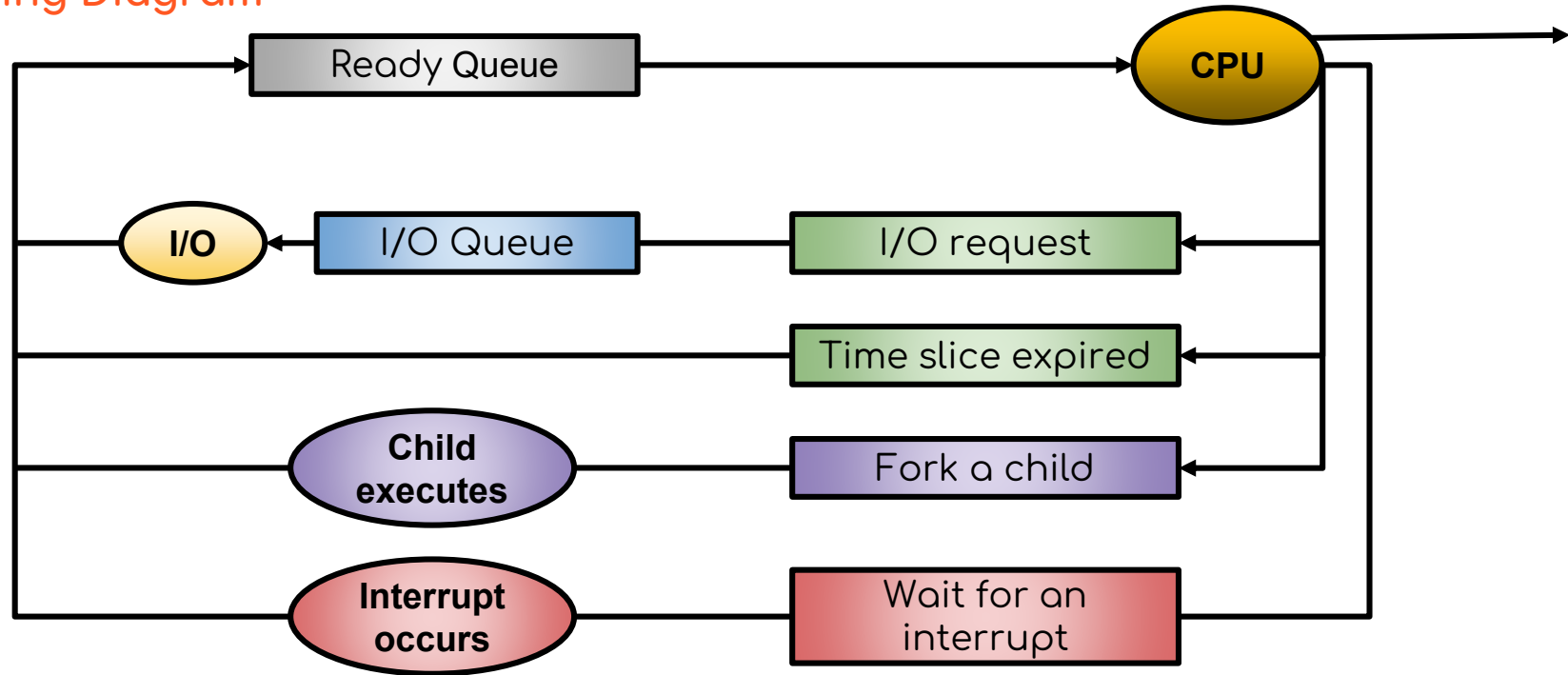
Process Scheduling Queues

- OS maintains scheduling queues of process
1. **Job queue** - Set of all processes in the system
 2. **Ready queue** - set of processes residing in the main memory, ready and waiting to execute.
 3. **Device queue** - Set of processes waiting for an IO device

Processes migrate among various queues

Process Scheduling Queues

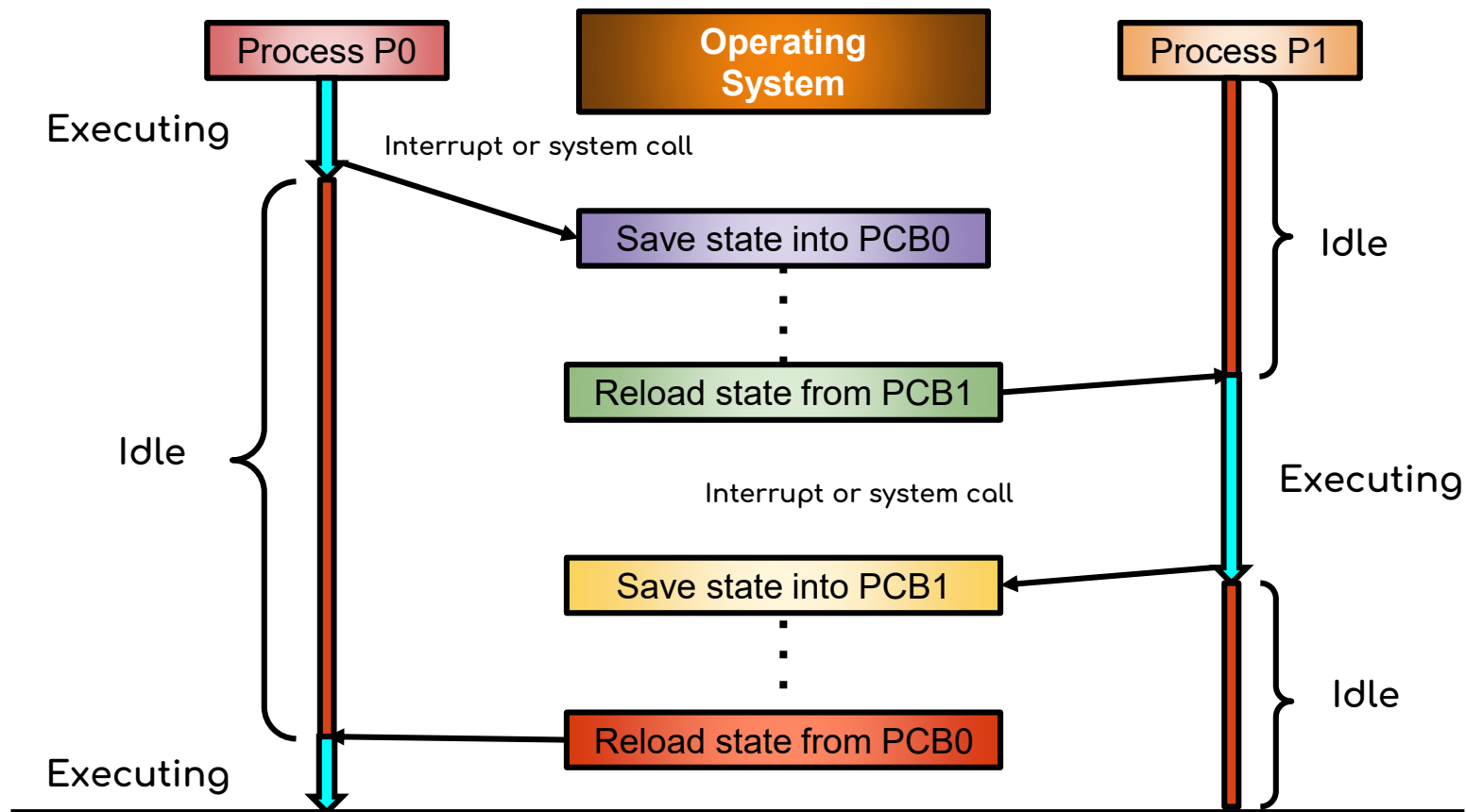
Queuing Diagram



Process - Context switching

- When CPU switches to another process, the system must save the state of the old process and load the saved state of new process via a **context switch**
- Context of the process is represented by PCB
- Context switch time is **pure overhead**, the system does no useful work while switching
- More complex OS and PCB implies more overheads of switch
- Time depends on hardware support - some hardware provides multiple set of registers so that multiple process contexts loaded/stored

Process - Context switching



CPU bound vs IO bound process

- A **CPU-bound** process requires more CPU time
- Spends more time in the running state.
- CPU bound means the program is bottlenecked by the CPU
- Spends more time doing computations
- In a CPU-bound environment, most times, the processor is the only component being used for execution.
- An **IO bound** process requires less CPU time
- An I/O-bound process spends more time in the waiting state.
- I/O bound means the program is bottlenecked by the IO operations
- Spends more time on IO
- I/O bound operations are characterized by many and fewer CPU bursts during execution



Schedulers



Short term Scheduler

1. Selects which process to be executed next and allocates CPU
2. Sometimes only scheduler
3. Short term scheduler is invoked frequently (in milliseconds), must be invoked very fast
4. The choices of the short term scheduler are very important. If it selects a process with a long burst time, then all the processes after that will have to wait for a long time in the ready queue.
5. This is known as starvation and it may happen if a wrong decision is made by the short-term scheduler.

Schedulers



Long term Scheduler

1. Selects the processes from the storage pool in the secondary memory and loading them into the ready queue in the main memory for execution.
2. The long-term scheduler controls the degree of multiprogramming.
3. It must select a careful mixture of I/O bound and CPU bound processes to yield optimum system throughput.
4. If it selects too many CPU bound processes then the I/O devices are idle and if it selects too many I/O bound processes then the processor has nothing to do.

Schedulers

Medium term Scheduler

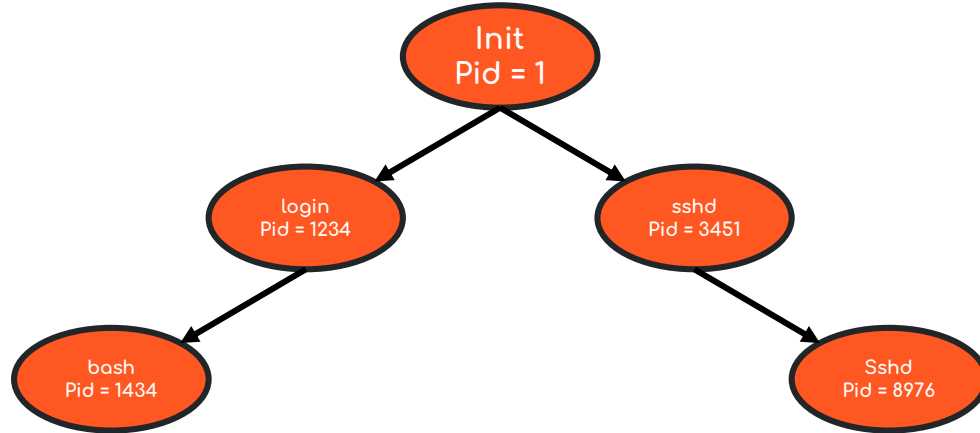
1. Medium-term scheduling involves swapping out a process from main memory.
2. The process can be swapped in later from the point it stopped executing.
3. This can also be called as suspending and resuming the process and is done by the medium-term scheduler.

Operation of Processes

- Process Creation
- Process Termination

Process Creation

- A process may create other processes
- Parent process creates children processes, which in turn create other processes, forming a tree of processes
- Generally a process is identified and managed via process identifier (PID)



A tree of processes in Unix

Process Creation

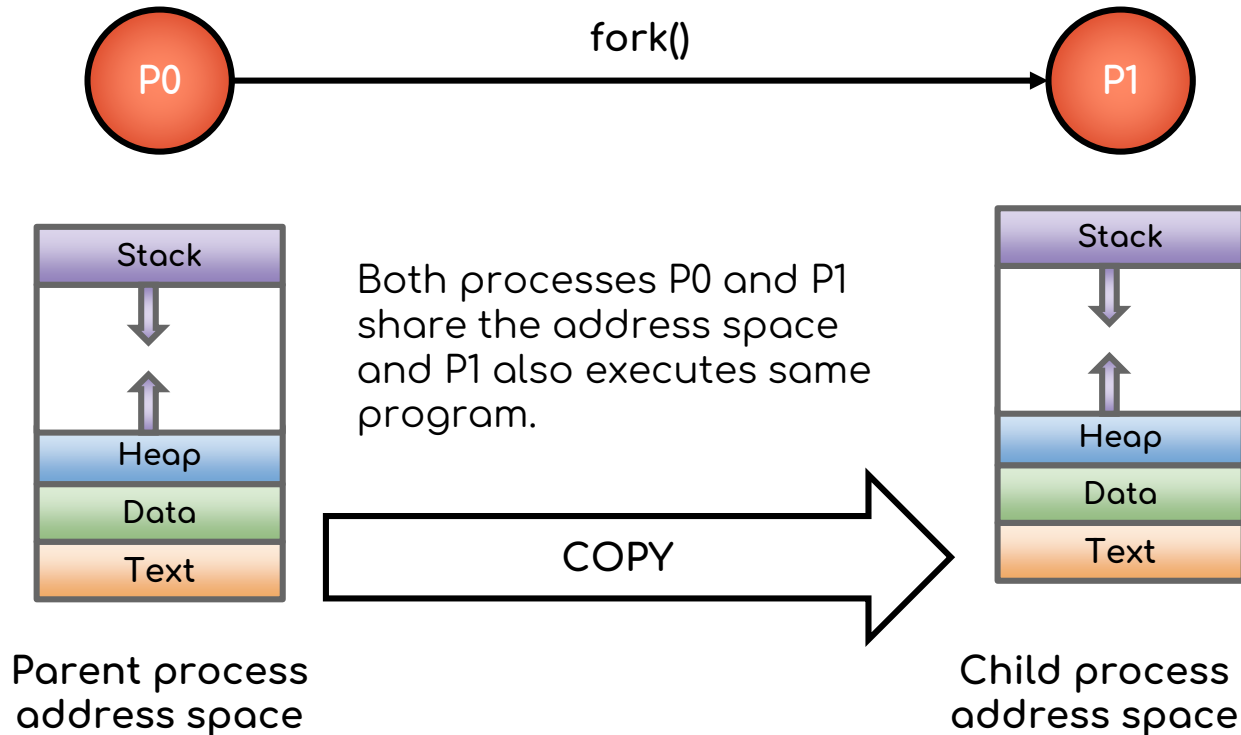
- Fork system call is used for creating a new process, which is called *child process*, which runs concurrently with the process that makes the fork() call (parent process).
- After a new child process is created, both processes will execute the next instruction following the fork() system call.
- A child process uses the same pc(program counter), same CPU registers, same open files which use in the parent process.
- It takes no parameters and returns an integer value. Below are different values returned by fork().

Negative Value: creation of a child process was unsuccessful.

Zero: Returned to the newly created child process.

Positive value: Returned to parent or caller. The value contains process ID of newly created child process.

Process Creation - fork()

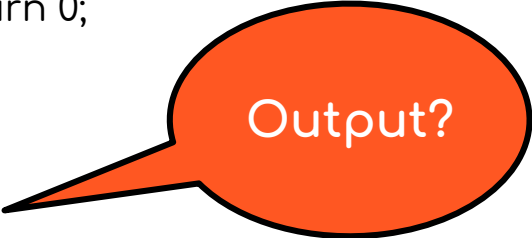


Process Creation - fork() program

```
1. #include <stdio.h>
2. #include <sys/types.h>
3. #include <unistd.h>
4. int main()
5. {
6.     // make two process which run same
7.     // program after this instruction
8.     fork();
9.     printf("Hello world!\n");
10.    return 0;
11. }
```

Output: -
Hello world!
Hello world!

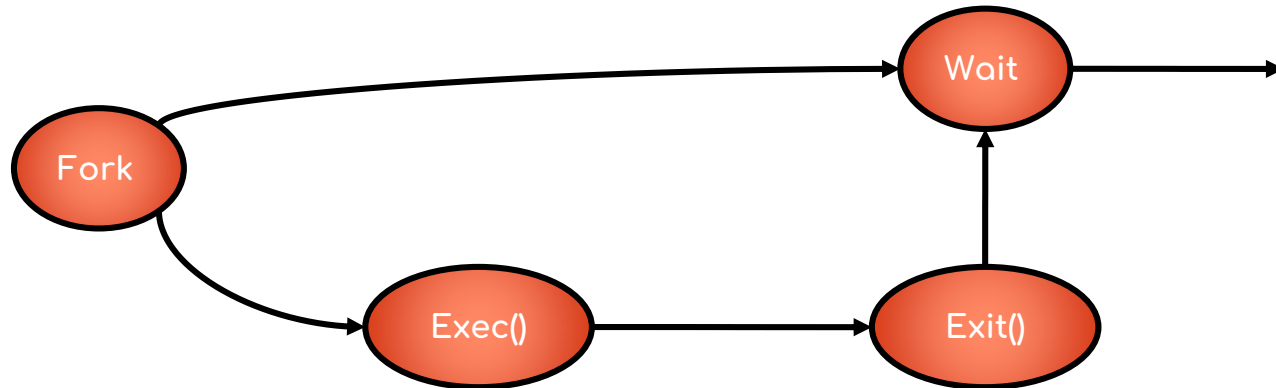
```
1. #include <stdio.h>
2. #include <sys/types.h>
3. int main()
4. {
5.     fork();
6.     fork();
7.     fork();
8.     printf("hello\n");
9.     return 0;
10. }
```



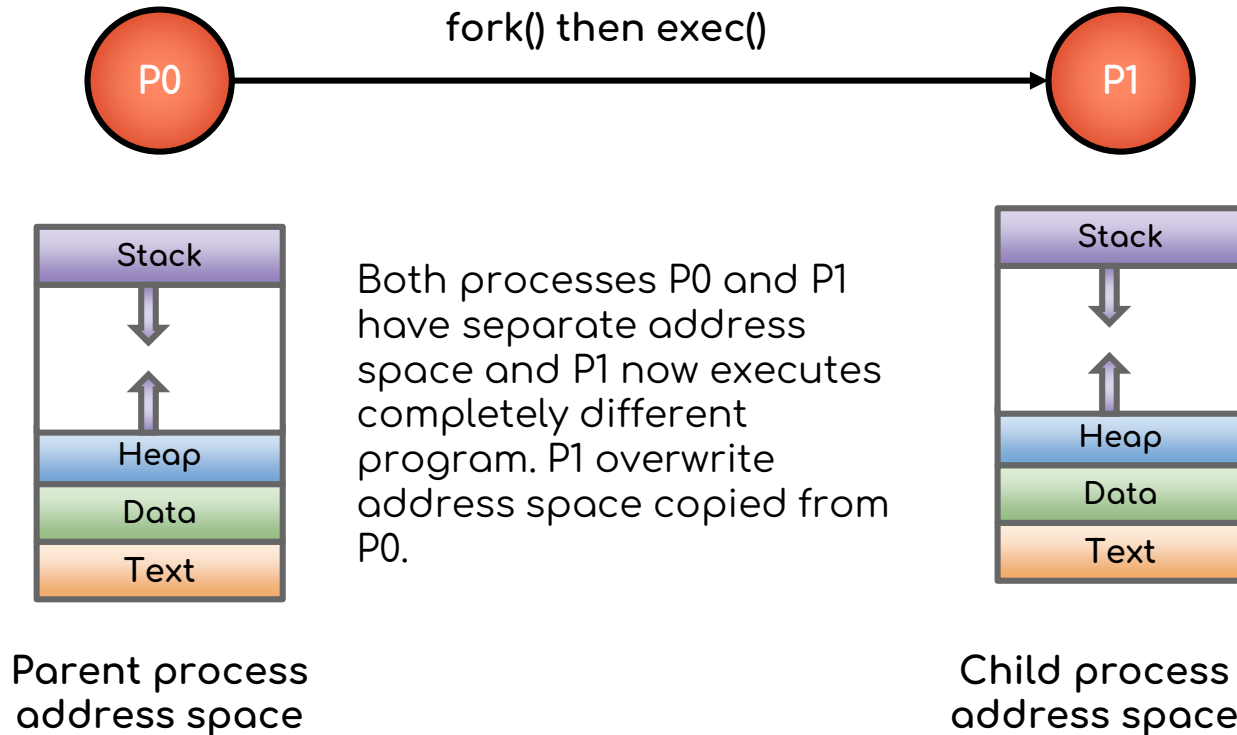
Output?

Process Creation - fork() and exec()

- fork() system call is used to create a new process
- exec() system call is used after a fork() replaces the process's memory space with a new program
- Using fork() a child is duplicate of the parents address space
- Using fork() A child loads program into address space



Process Creation - fork() and exec()



Process Creation - Exec() program

```
1. //EXEC.c
2. #include<stdio.h>
3. #include<unistd.h>
4. int main()
5. {
6.     int i;
7. printf("I am EXEC.c called by
   execvp() ");
8. printf("\n");
9. return 0;
10.}
```

COMPILE AND CREATE EXECUTABLE

```
gcc EXEC.c -o EXEC
```

```
1. #include<stdio.h>
2. #include<stdlib.h>
3. #include<unistd.h>
4. int main()
5. {
6. //A null terminated array of character
7. //pointers
8. char *args[]={"./EXEC",NULL};
9. execvp(args[0],args);
10.printf("Ending-----");
11.     return 0;
12.}
```

Compile and Run this program

```
gcc execDemo.c -o execDemo
```

Process Termination

- A process terminates when it executes its final statement and ask OS to delete it by using `exit()` system call
- All the resources allocated to process gets deallocated by the operating system
- A parent may terminate the execution of child process using abort system call, some reasons to do so are
 1. Child has exceeded allocated resources
 2. Task assigned to child is no longer required
 3. The parent is exiting and OS does not allow a child to continue if the parent terminates.

Process Termination

1. A parent process may wait till child process terminates using **wait() system call**. The call returns status and pid of terminated process.

```
pid = wait(&status);
```

1. A process that has finished the execution but still has an entry in the process table called as **Zombie process**
2. An **orphan process** is a computer process whose parent process has finished or terminated, though it (child process) remains running itself.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t child_pid;

    // Create a child process
    child_pid = fork();

    if (child_pid == 0) {
        printf("Child process\n");
    } else if (child_pid > 0) {
        printf("Parent process\n");
    } else {
        perror("Fork failed");
    }

    return 0;
}
```

Parent process
Child process

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t child_pid;

    // Create a child process
    child_pid = fork();

    if (child_pid == 0) {
        printf("Child process: PID=%d\n", getpid());
    } else if (child_pid > 0) {
        printf("Parent process: PID=%d, Child PID=%d\n", getpid(), child_pid);
    } else {
        perror("Fork failed");
    }

    return 0;
}
```

```
Parent process: PID=1862, Child PID=1863
Child process: PID=1863
```

```

#include <stdio.h>

int main() {
    int i;

    for (i = 0; i < 3; i++) {
        pid_t child_pid = fork();

        if (child_pid == 0) {
            printf("Child process %d: PID=%d\n", i + 1, getpid());
            break; // Each child process should exit the loop
        } else if (child_pid < 0) {
            perror("Fork failed");
        }
    }

    if (getpid() != 1) {
        // This code is executed only by the parent process
        printf("Parent process: PID=%d\n", getpid());
    }

    return 0;
}

```

```

Parent process: PID=1926
Child process 1: PID=1927
Parent process: PID=1927
Child process 2: PID=1928
Parent process: PID=1928
Child process 3: PID=1929
Parent process: PID=1929

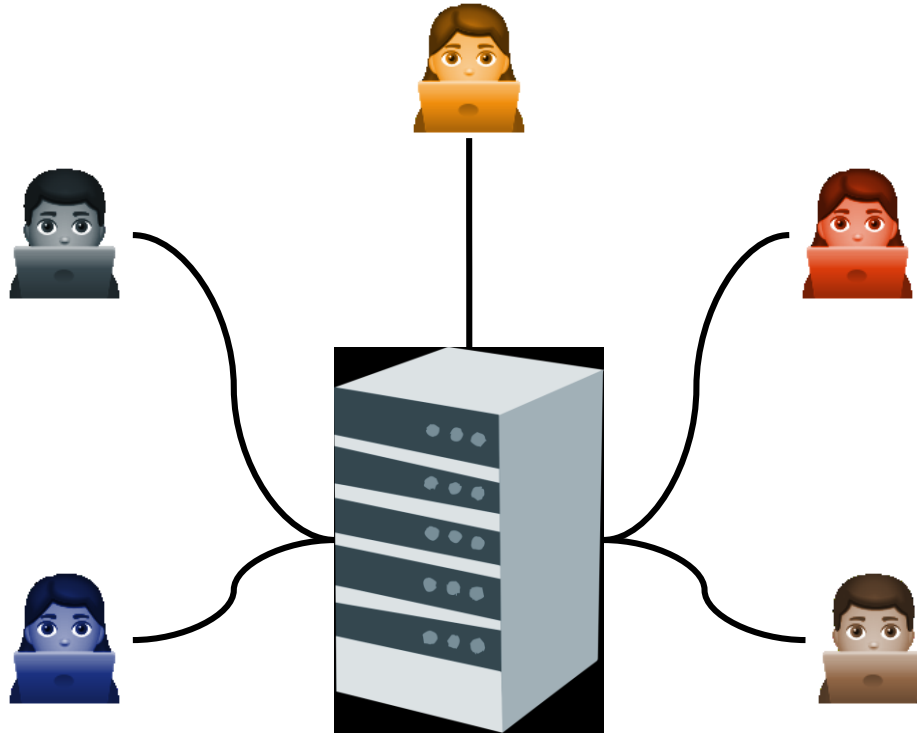
```


Threads

Threads

- The process model so far considered that a process is a program that performs a single thread of execution.
- For example, if a process is running a word-processor program, a single thread of instructions is being executed.
- This single thread of control allows the process to perform only one task at a time.
- For example, user could not simultaneously type in characters and run spell checker within the same process.
- Many modern OS have extended process concept to allow a process to have multiple threads of execution thus user will be perform more than one task at a time.
- Thread is a lightweight process, is a basic unit of CPU utilization. (tid, PC, register set, stack)

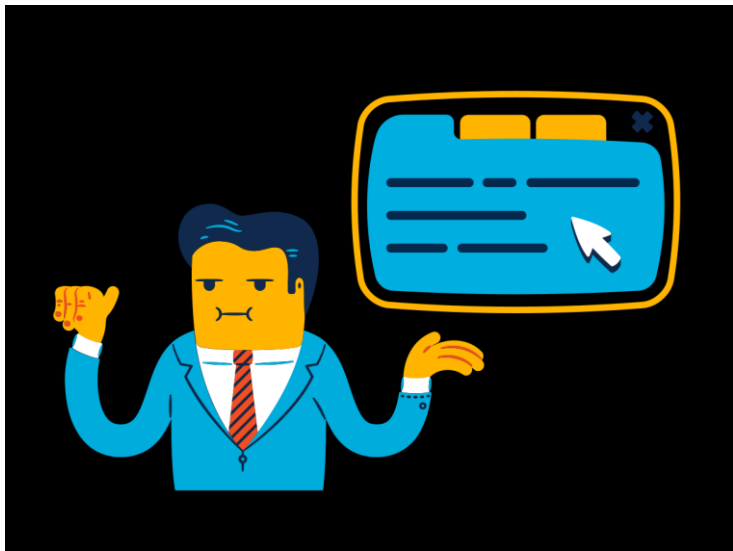
Web server example



Web server

- If we handle one user at a time it may lead to starvation to others
- If for each user a separate process is created:-
- Each process contains data, text, stack and heap region
- If multiple processes get created it becomes very heavy to operate; takes more time.

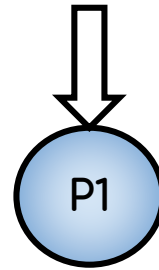
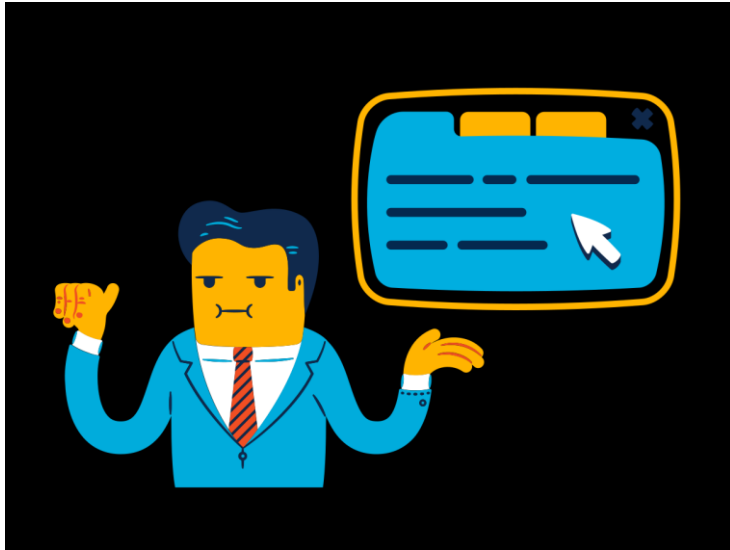
Motivation for threads



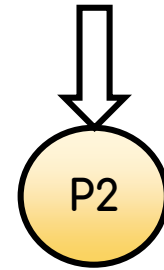
Let's say there are multiple tasks with a computer application

1. Displaying graphics,
2. Reading keystrokes from the user
3. Spell check and Grammar
4. Fetch data from web
5. Answer to a network request

Motivation for threads



Fetch data



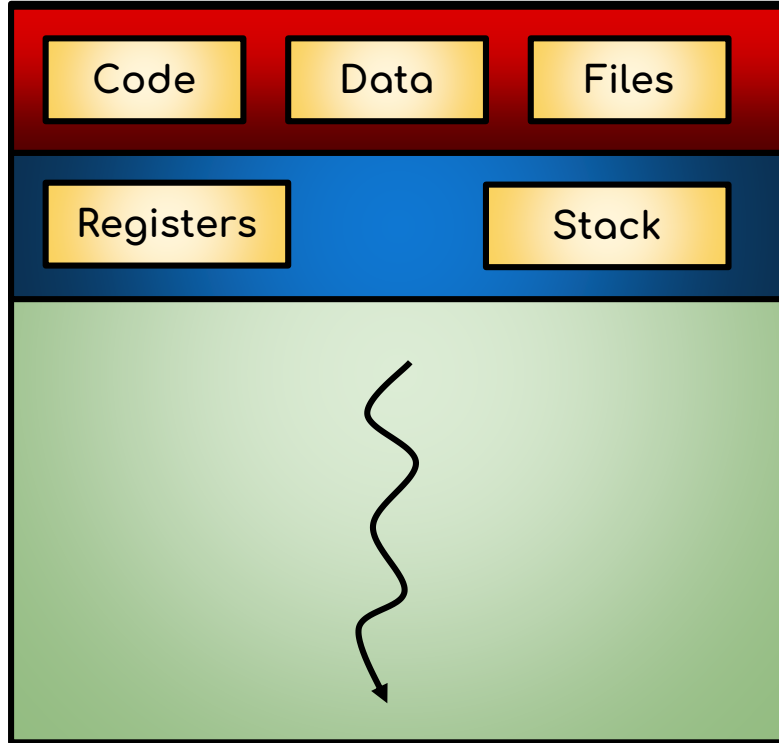
Spell Checking

Two separate processes created for each operation

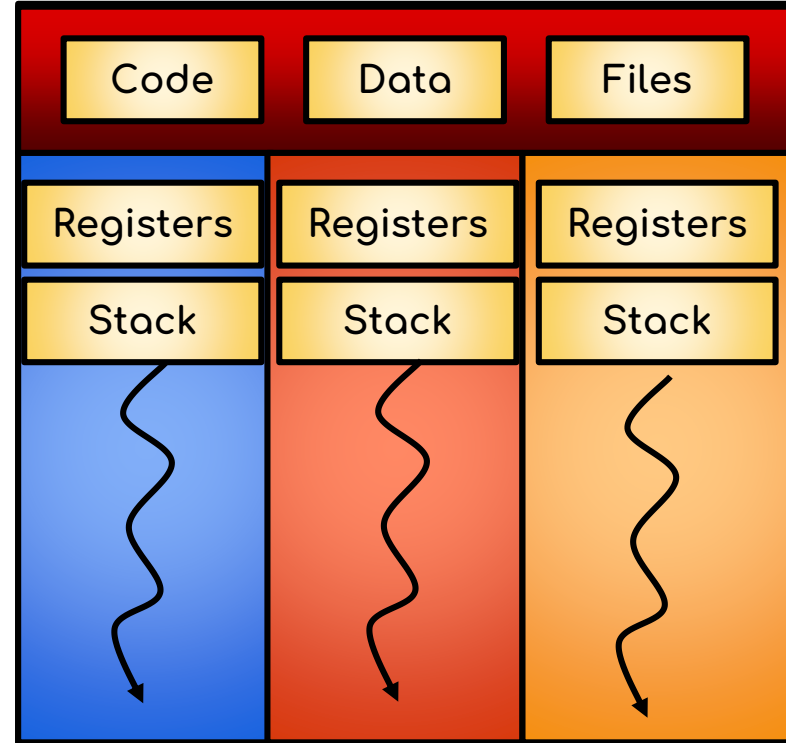
Process creation is heavy weight - code, data, stack segments, hence switching is also slow.

- If parent child relation between processes then shared memory needs to be created which is limited

Single vs multi threaded process

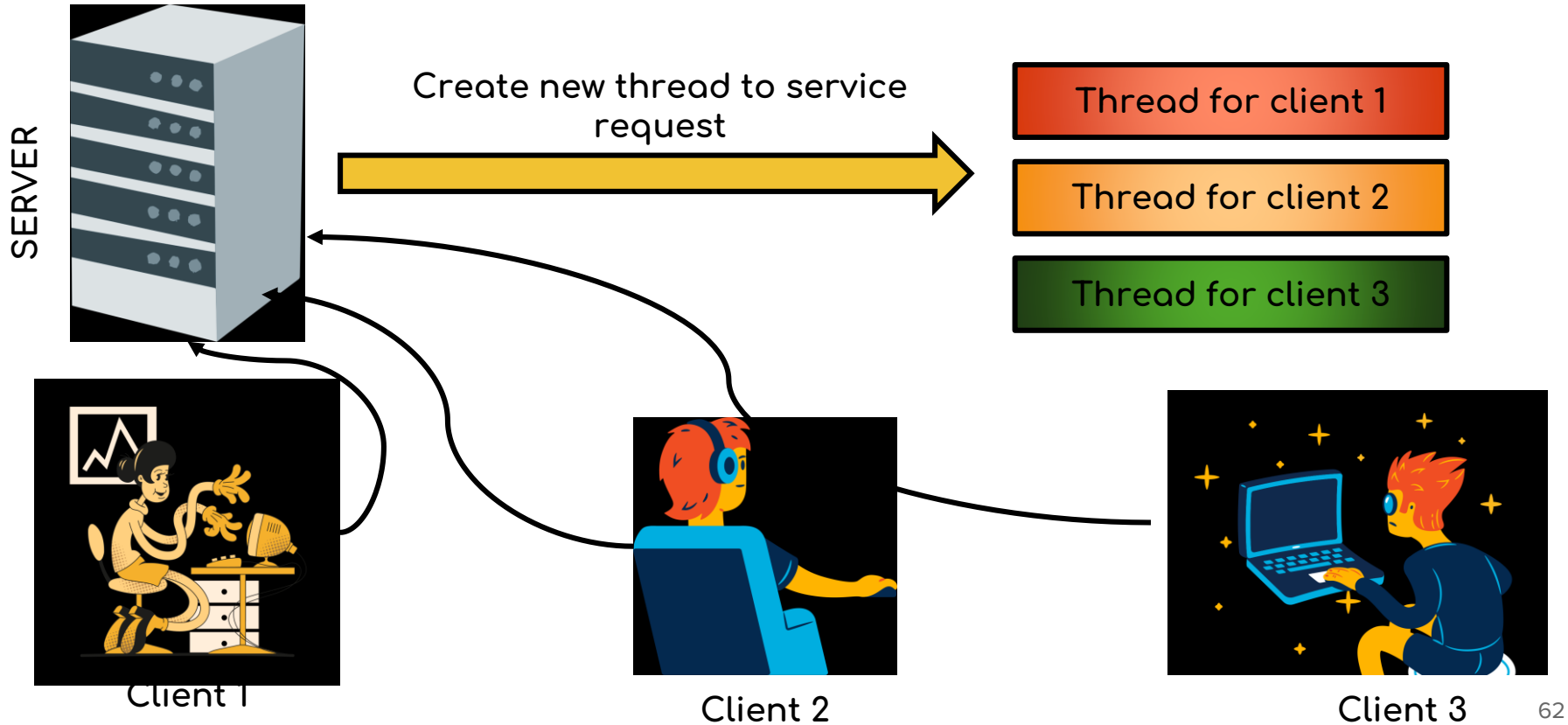


Single threaded
process



Multi-threaded process

Multi-threaded server architecture



Benefits of multi-threading

- 1 Responsiveness - May allow continued execution if part of process is blocked, especially important for user interfaces
- 2 Resource sharing - Thread share resources of process easier than shared memory.
- 3 Economy - Cheaper than process creation, thread switching overheads are lesser than context switch process overhead
- 4 Scalability – process with multiple threads can take advantage of multiprocessor architecture. A single thread process can run on only one processor, regardless how many are available

User threads and kernel threads

Support for threads also provided at two levels

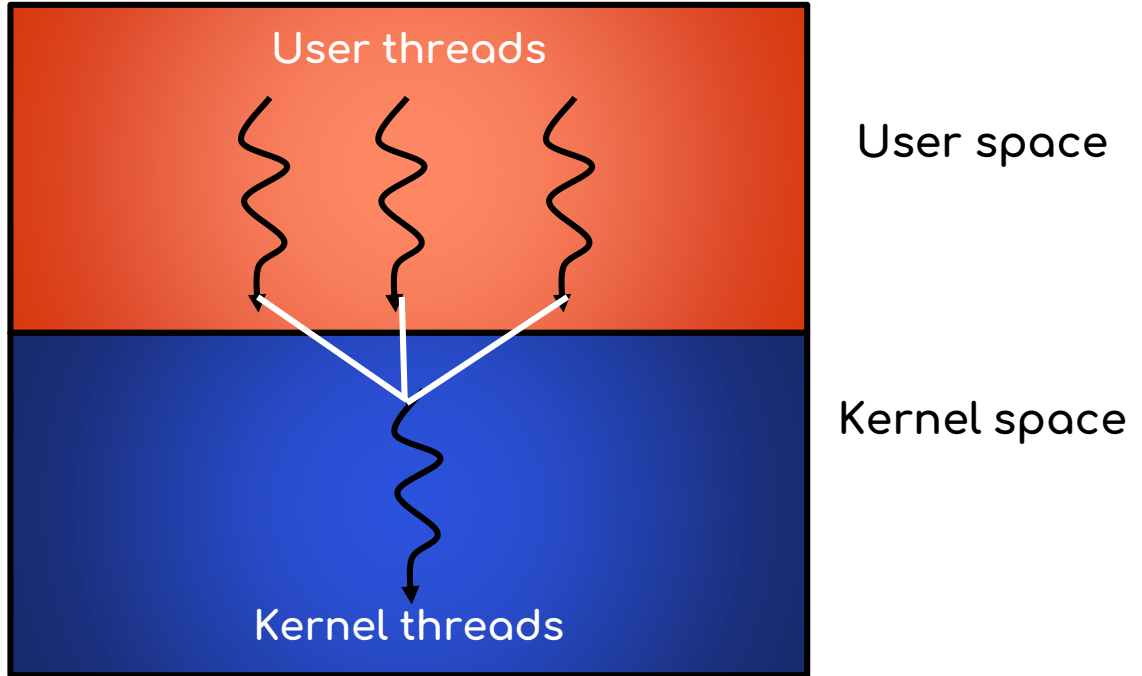
- **User threads** - Supported above the kernel and are managed without support of the kernel.
Generally handled by a programmer or by a threads library
- **Kernel threads** - Supported and managed directly by OS

Relationship between user and kernel threads

- Many-to-one
- One-to-one
- Many-to-many

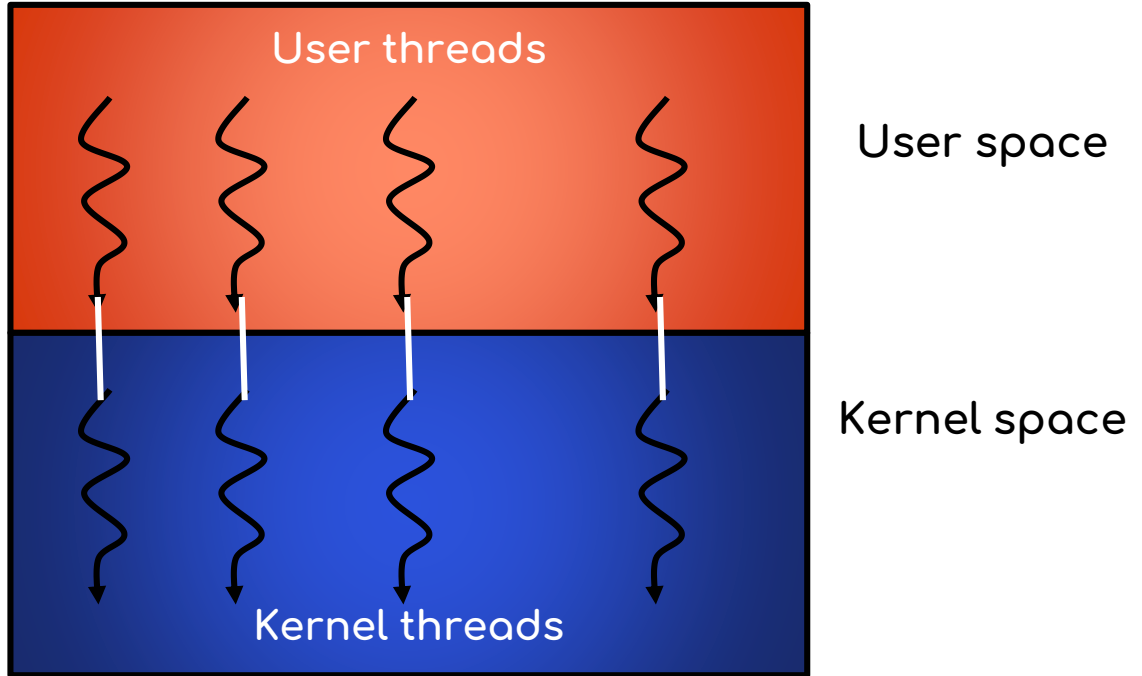
Relationship between user and kernel threads

- Many-to-one
- One-to-one
- Many-to-many
- Many user level threads mapped to one kernel thread
- One thread blocking cause all to block; entire process will be blocked.
- Multiple threads may not run in parallel because only one may be in kernel at a time



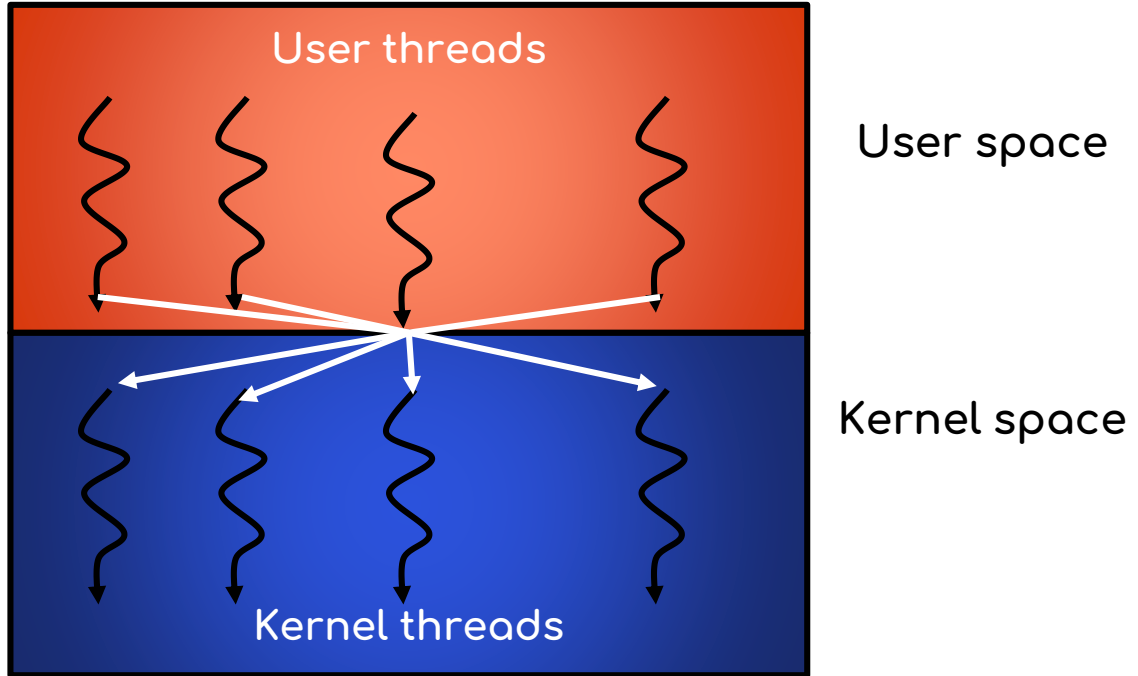
Relationship between user and kernel threads

- Many-to-one
- **One-to-one**
- Many-to-many
- Whenever a new thread is created in user space, a new thread is created in kernel space
- Each user thread, kernel thread handles system calls
- Less blocking at OS level



Relationship between user and kernel threads

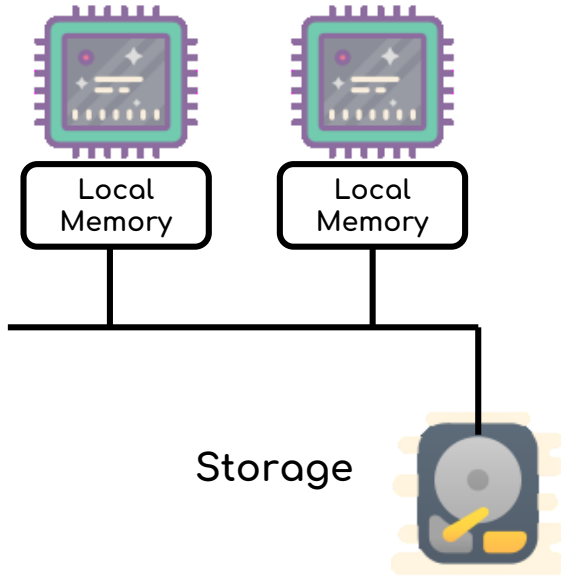
- Many-to-one
- One-to-one
- **Many-to-many**
- Allows many user threads mapped to many kernel threads
- Allows OS to create sufficient number of kernel threads



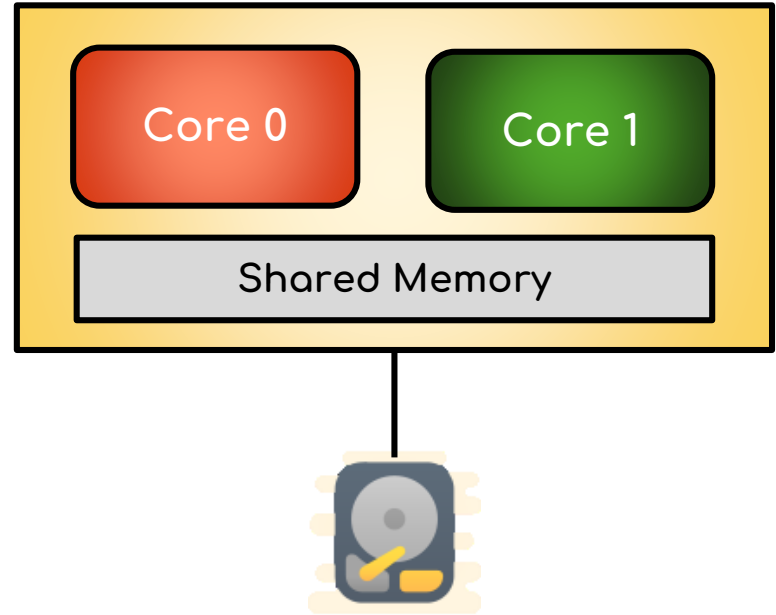
Thread Libraries

- Thread libraries provide programmer with API creating and managing threads
- Two primary ways of implementing
 1. Library entirely in user space
 2. Kernel library supported by OS
- Three primary thread libraries
 1. POSIX Pthreads
 2. Windows threads
 3. Java threads

Multi-CPU vs Multi-core



- Multiple CPUs placed in a computer
- Each processor is having its own local memory



- Multiple cores placed on a single processor chip
Each core appears as separate CPU for OS

Serial vs concurrency vs parallelism



Host

Serial

- Host is talking with one guest at a time
- While host talking with one guest other guests wait till his/her chance
- Host communicates with a guest **till it finishes**, it doesn't switch to another guest in between

Serial vs concurrency vs parallelism



Host

Concurrent

- Host is talking with one guest at a time for one minute
- While host talking with one guest other guests wait till his/her chance
- Host switch to another guest after one minute
- Each guest will get time to communicate with host for some time

Serial vs concurrency vs parallelism



Host

Parallel

- Multiple host copies are available
- Each guest is communicating with host without any intervention

Serial vs concurrency vs parallelism



Host

Concurrency +
Parallelism

- Multiple host copies (ghost :-) are available
- Each guest is communicating with two host concurrently